

Optimisation du traitement des requêtes cli

Back Office Customer Care — Moov Africa

Analyse de données | Data Science Project

Champ	Détail
Projet	Optimisation Back Office Customer Care
Entreprise	Moov Africa
Auteur	Data Analyst / Data Scientist
Date	2 0 2 4
Version	2 . 0 — Enrichie & Documentée
Outils	Python · Pandas · Seaborn · Plotly · Scikit-learn
Livrable BI	Dashboard Power BI multi-vues

Objectifs du projet

- 1 . **Comprendre** la nature et le volume des requêtes clients traitées par le Back Office
 - 2 . **Identifier** les goulots d'étranglement et les catégories les plus chronophages
 - 3 . **Mesurer** la performance des agents et le respect des SLA
 - 4 . **Modéliser** les délais de traitement pour anticiper les charges
 - 5 . **Produire** des données propres et structurées pour le Dashboard Power BI
-

Structure du Notebook

01. Configuration & Imports
02. Chargement & Inspection des données
03. Nettoyage & Qualité des données
04. Analyse Exploratoire (EDA) — Tickets
05. Analyse Exploratoire (EDA) — Agents
06. KPIs Métier & SLA
07. Feature Engineering
08. Modélisation (Clustering + Régression)
09. Export pour Power BI
10. Conclusions & Recommandations

0 1 . Configuration & Imports

In [1]:

```
# -----  
# IMPORTS – Bibliothèques utilisées dans ce notebook  
# -----  
  
# Manipulation des données  
import pandas as pd  
import numpy as np  
  
# Visualisation statique  
import matplotlib.pyplot as plt  
import matplotlib.dates as mdates  
import seaborn as sns  
  
# Visualisation interactive  
import plotly.express as px  
import plotly.graph_objects as go  
from plotly.subplots import make_subplots  
  
# Machine Learning  
from sklearn.preprocessing import LabelEncoder, StandardScaler  
from sklearn.cluster import KMeans  
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor  
from sklearn.linear_model import LinearRegression  
from sklearn.model_selection import train_test_split, cross_val_score  
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score  
from sklearn.pipeline import Pipeline  
  
# Statistiques  
from scipy import stats  
  
# Utilitaires  
import warnings  
import os  
from datetime import datetime, timedelta  
  
warnings.filterwarnings("ignore")  
  
# -----  
# CONFIGURATION GLOBALE  
# -----  
  
# Palette couleurs Moov Africa (bleu + orange)  
MOOV_BLUE = "#003399"  
MOOV_ORANGE = "#FF6600"  
MOOV_LIGHT = "#E8F0FF"  
MOOV_GRAY = "#6C757D"  
MOOV_SUCCESS = "#28A745"  
MOOV_DANGER = "#DC3545"  
MOOV_WARN = "#FFC107"  
  
PALETTE_MOOV = [  
    MOOV_BLUE,  
    MOOV_ORANGE,  
    "#0055CC",  
    "#FF8833",  
    "#0077FF",  
    "#FFAA55",  
    "#003380",  
    "#CC5500",  
]
```

```

]

# Style Matplotlib
plt.rcParams.update(
    {
        "figure.figsize": (12, 5),
        "figure.dpi": 120,
        "axes.spines.top": False,
        "axes.spines.right": False,
        "axes.titlesize": 14,
        "axes.titleweight": "bold",
        "axes.labelsize": 11,
        "xtick.labelsize": 10,
        "ytick.labelsize": 10,
        "legend.fontsize": 10,
        "font.family": "sans-serif",
        "axes.grid": True,
        "grid.alpha": 0.3,
        "grid.linestyle": "--",
    }
)

sns.set_theme(style="whitegrid", palette=PALETTE_MOOV)

print("Configuration chargée avec succès")
print(f"    Pandas      : {pd.__version__}")
print(f"    NumPy       : {np.__version__}")
print(f"    Seaborn     : {sns.__version__}")

```

Configuration chargée avec succès

```

Pandas      : 2.3.3
NumPy       : 2.2.6
Seaborn     : 0.13.2

```

0 2. Chargement & Inspection des données

Note : Remplace les chemins de fichiers ci-dessous par les chemins réels de tes dataset
Le notebook supporte les formats CSV, Excel (.xlsx) et JSON.

```

In [2]: # -----
# CHARGEMENT DES DONNÉES RÉELLES – Call Center Moov Africa
# -----

EXCEL_PATH = "DatasetCallCenter - 2026-05-05T082442.350.xlsx"

df_raw = pd.read_excel(EXCEL_PATH)
print(f"Chargé : {EXCEL_PATH}")
print(f"    {df_raw.shape[0]:,} lignes x {df_raw.shape[1]} colonnes")

# Renommage des colonnes (évite les apostrophes spéciales)
df_raw.columns = [
    "titre_demande", "numero_incident", "cree_le", "cree_par",
    "priorite", "site", "distributeur", "espace_service",
    "file_attente", "client", "numero_interaction", "statut", "date_clotu
]

# -----
# MAPPING – Colonnes réelles vers structure notebook

```

```

# -----

df_tickets = pd.DataFrame()

df_tickets["ticket_id"] = df_raw["numero_incident"]
df_tickets["date_creation"] = pd.to_datetime(df_raw["cree_le"])
df_tickets["date_cloture"] = pd.to_datetime(df_raw["date_cloture"])
df_tickets["delai_traitement_h"] = (
    (df_tickets["date_cloture"] - df_tickets["date_creation"])
    .dt.total_seconds() / 3600
).clip(lower=0).round(2)

df_tickets["categorie"] = df_raw["titre_demande"].str.split("|").str[0]
df_tickets["sous_categorie"] = df_raw["titre_demande"].str.split("|").str[1]
df_tickets["agent_id"] = df_raw["cree_par"]
df_tickets["canal"] = df_raw["file_attente"].fillna("Non affecté")
df_tickets["priorite"] = df_raw["priorite"]
df_tickets["statut"] = df_raw["statut"]
df_tickets["client"] = df_raw["client"]
df_tickets["region"] = df_raw["site"].fillna("Call Center")

# Colonnes absentes du dataset réel
df_tickets["satisfaction_score"] = np.nan
df_tickets["nb_relances"] = 0
df_tickets["montant_litige"] = 0
df_tickets["premiere_resolution"] = np.nan

# -----
# DATASET AGENTS – construit depuis les tickets
# -----

agents_list = df_tickets["agent_id"].dropna().unique().tolist()

def agent_equipe(a):
    sub = df_tickets[df_tickets["agent_id"] == a]["canal"]
    return sub.mode()[0] if not sub.empty else "Non affecté"

def agent_specialite(a):
    sub = df_tickets[df_tickets["agent_id"] == a]["categorie"]
    return sub.mode()[0] if not sub.empty else "N/A"

df_agents = pd.DataFrame({
    "agent_id" : agents_list,
    "nom_agent" : agents_list,
    "equipe" : [agent_equipe(a) for a in agents_list],
    "anciennete_mois" : 0,
    "specialite" : [agent_specialite(a) for a in agents_list],
    "taux_resolution" : 0.0,
    "objectif_tickets_j" : 0,
    "score_qualite" : 0.0,
})

print(f"\nDataset Tickets : {df_tickets.shape[0]:,} lignes x {df_tickets.shape[1]:,} colonnes")
print(f"Dataset Agents : {df_agents.shape[0]:,} agents identifiés")
print(f"Période : {df_tickets['date_creation'].min().date()} -> {df_tickets['date_cloture'].max().date()}")
print(f"\nTop catégories :")
print(df_tickets["categorie"].value_counts().head(8).to_string())

```

Chargé : DatasetCallCenter - 2026-05-05T082442.350.xlsx
2,287 lignes x 13 colonnes

Dataset Tickets : 2,287 lignes x 16 colonnes

Dataset Agents : 91 agents identifiés

Période : 2026-01-01 -> 2026-05-05

Top catégories :

categorie	
Réclamations FTTH	988
Réclamations sur le Réseau & les services voix/SMS	374
Réclamations Moov Money	316
Contestations crédit et réclamations sur le rechargement	143
Opérations Moov Money	130
Demande FTTH	122
Réclamations SVA	56
Réclamations sur les services Data	52

0 3. 🧹 Nettoyage & Qualité des données

Avant toute analyse, nous vérifions la **qualité du dataset** : valeurs manquantes, doublons, types données et cohérence.

```
In [3]: # -----  
# AUDIT DE QUALITÉ DES DONNÉES  
# -----  
  
def audit_qualite(df: pd.DataFrame, nom: str = "DataFrame") -> pd.DataFrame:  
    """Génère un rapport de qualité complet pour un DataFrame."""  
    rapport = pd.DataFrame(  
        {  
            "Type": df.dtypes,  
            "Non-nuls": df.count(),  
            "Nuls": df.isnull().sum(),  
            "Nuls_%": (df.isnull().sum() / len(df) * 100).round(2),  
            "Doublons_col": [df[c].duplicated().sum() for c in df.columns],  
            "Cardinalité": df.nunique(),  
            "Exemple": [  
                df[c].dropna().iloc[0] if df[c].dropna().shape[0] > 0 else  
                for c in df.columns  
            ],  
        }  
    )  
  
    print(f"\n{'='*60}")  
    print(f"  RAPPORT QUALITÉ — {nom}")  
    print(f"{'='*60}")  
    print(f"  Dimensions      : {df.shape[0]:,} lignes × {df.shape[1]} colonnes")  
    print(f"  Doublons        : {df.duplicated().sum()} lignes identiques")  
    print(f"  Mémoire         : {df.memory_usage(deep=True).sum() / 1024**2:.2f} Mo")  
    print(f"{'='*60}\n")  
    return rapport  
  
rapport_tickets = audit_qualite(df_tickets, "Tickets Back Office")  
rapport_tickets
```

RAPPORT QUALITÉ – Tickets Back Office

Dimensions : 2,287 lignes × 16 colonnes
 Doublons : 0 lignes identiques
 Mémoire : 1.92 MB

Out[3]:

	Type	Non-nuls	Nuls	Nuls_%	Doublons_col	(
ticket_id	object	2 2 8 7	0	0.0	0	
date_creation	datetime 6 4 [ns]	2 2 8 7	0	0.0	6	
date_cloture	datetime 6 4 [ns]	2 2 8 7	0	0.0	5 3	
delai_traitement_h	float 6 4	2 2 8 7	0	0.0	1 3 3 5	
categorie	object	2 2 8 7	0	0.0	2 2 6 7	
sous_categorie	object	2 2 8 7	0	0.0	2 1 9 3	
agent_id	object	2 2 8 7	0	0.0	2 1 9 6	
canal	object	2 2 8 7	0	0.0	2 2 8 0	
priorite	object	2 2 8 7	0	0.0	2 2 8 4	
statut	object	2 2 8 7	0	0.0	2 2 8 6	
client	object	2 2 8 7	0	0.0	8 1 9	
region	object	2 2 8 7	0	0.0	2 2 8 6	
satisfaction_score	float 6 4	0 2 2 8 7	1 0 0.0	2 2 8 6		
nb_relances	int 6 4	2 2 8 7	0	0.0	2 2 8 6	
montant_litige	int 6 4	2 2 8 7	0	0.0	2 2 8 6	
premiere_resolution	float 6 4	0 2 2 8 7	1 0 0.0	2 2 8 6		

In [4]:

```
# -----
# NETTOYAGE – Traitement des anomalies
# -----

df_clean = df_tickets.copy()

# 1. Suppression des doublons
n_avant = len(df_clean)
df_clean.drop_duplicates(subset=["ticket_id"], inplace=True)
print(f"✅ Doublons supprimés : {n_avant - len(df_clean)}")

# 2. Conversion des types temporels
df_clean["date_creation"] = pd.to_datetime(df_clean["date_creation"])

# 3. Extraction de variables temporelles utiles
```

```

df_clean["annee"] = df_clean["date_creation"].dt.year
df_clean["mois"] = df_clean["date_creation"].dt.month
df_clean["mois_nom"] = df_clean["date_creation"].dt.strftime("%b %Y")
df_clean["semaine"] = df_clean["date_creation"].dt.isocalendar().week.astype(int)
df_clean["jour_semaine"] = df_clean["date_creation"].dt.day_name()
df_clean["heure"] = df_clean["date_creation"].dt.hour
df_clean["est_weekend"] = df_clean["date_creation"].dt.dayofweek >= 5

# 4. Nettoyage des valeurs aberrantes (délais)
Q1 = df_clean["delai_traitement_h"].quantile(0.25)
Q3 = df_clean["delai_traitement_h"].quantile(0.75)
IQR = Q3 - Q1
born_sup = Q3 + 3 * IQR
n_outliers = (df_clean["delai_traitement_h"] > born_sup).sum()
print(f"✅ Outliers délais détectés : {n_outliers} (seuil : {born_sup:.1f})

# Conservation avec flag plutôt que suppression
df_clean["est_outlier_delai"] = df_clean["delai_traitement_h"] > born_sup

# 5. Standardisation des catégories textuelles
df_clean["categorie"] = df_clean["categorie"].str.strip()
df_clean["statut"] = df_clean["statut"].str.strip()

# 6. Flag SLA
SLA_HEURES = 24
df_clean["respecte_sla"] = df_clean["delai_traitement_h"] <= SLA_HEURES
df_clean["depassement_sla_h"] = (df_clean["delai_traitement_h"] - SLA_HEURES).clip(lower=0)

)

print(f"\n✅ Nettoyage terminé")
print(f"    Lignes finales      : {len(df_clean):,}")
print(f"    Taux SLA global      : {df_clean['respecte_sla'].mean()*100:.1f}%")
print(
    f"    Colonnes ajoutées : annee, mois, mois_nom, semaine, jour_semaine,
    heure, est_weekend, respecte_sla, depassement_sla_h
)

```

✅ Doublons supprimés : 0

✅ Outliers délais détectés : 137 (seuil : 82.5h)

✅ Nettoyage terminé

Lignes finales : 2,287

Taux SLA global : 78.3%

Colonnes ajoutées : annee, mois, mois_nom, semaine, jour_semaine, heure, est_weekend, respecte_sla, depassement_sla_h

0 4 . Analyse Exploratoire — Tickets

Nous analysons ici la **distribution et les tendances** des tickets clients : volumes, catégories, et temporalité.

In [5]:

```

# -----
# DISTRIBUTION DES VOLUMES PAR CATÉGORIE
# -----

vol_cat = df_clean["categorie"].value_counts().reset_index()
vol_cat.columns = ["categorie", "volume"]
vol_cat["pct"] = (vol_cat["volume"] / vol_cat["volume"].sum() * 100).round(1)

```

```

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Barplot horizontal
bars = axes[0].barh(
    vol_cat["categorie"],
    vol_cat["volume"],
    color=PALETTE_MOOV[: len(vol_cat)],
    edgecolor="white",
    height=0.7,
)
axes[0].set_title("Volume de tickets par catégorie", fontweight="bold")
axes[0].set_xlabel("Nombre de tickets")
for bar, pct in zip(bars, vol_cat["pct"]):
    axes[0].text(
        bar.get_width() + 20,
        bar.get_y() + bar.get_height() / 2,
        f"{pct}%",
        va="center",
        fontsize=9,
        color=MOOV_GRAY,
    )
axes[0].invert_yaxis()

# Pie chart
axes[1].pie(
    vol_cat["volume"],
    labels=vol_cat["categorie"],
    autopct="%1.1f%%",
    colors=PALETTE_MOOV[: len(vol_cat)],
    startangle=140,
    pctdistance=0.8,
    wedgeprops=dict(edgecolor="white", linewidth=1.5),
)
axes[1].set_title("Répartition par catégorie", fontweight="bold")

plt.suptitle(
    "Distribution des requêtes clients – Back Office Moov Africa",
    fontsize=14,
    fontweight="bold",
    y=1.02,
)
plt.tight_layout()
plt.savefig("output_distribution_categories.png", dpi=150, bbox_inches="tight")
plt.show()
print(
    f"\n🔍 Catégorie dominante : {vol_cat.iloc[0]['categorie']} ({vol_cat

```

🔍 Catégorie dominante : Réclamations FTTH (43.2%)

```

In [6]: # -----
# ÉVOLUTION TEMPORELLE DES VOLUMES
# -----

vol_mensuel = df_clean.groupby("mois_nom").size().reset_index(name="volume")
# Tri chronologique
vol_mensuel["date_sort"] = pd.to_datetime(vol_mensuel["mois_nom"], format="%Y-%m")
vol_mensuel = vol_mensuel.sort_values("date_sort")

```



```

fig = px.line(
    vol_mensuel,
    x="mois_nom",
    y="volume",
    title="📈 Évolution mensuelle du volume de tickets – Moov Africa Back
markers=True,
color_discrete_sequence=[MOOV_BLUE],
labels={"mois_nom": "Mois", "volume": "Nombre de tickets"},
)

fig.update_traces(line_width=2.5, marker_size=8)
fig.update_layout(
    title_font_size=14,
    plot_bgcolor="white",
    paper_bgcolor="white",
    xaxis=dict(showgrid=True, gridcolor="#F0F0F0"),
    yaxis=dict(showgrid=True, gridcolor="#F0F0F0"),
    hovermode="x unified",
)
fig.show()

```

In [7]:

```

# -----
# HEATMAP – ACTIVITÉ PAR JOUR & HEURE
# -----

jours_ordre = [
    "Monday",
    "Tuesday",
    "Wednesday",
    "Thursday",
    "Friday",
    "Saturday",
    "Sunday",
]

jours_fr = ["Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi",
            "Dimanche"]

heatmap_data = df_clean.groupby(["jour_semaine", "heure"]).size().unstack
heatmap_data = heatmap_data.reindex(jours_ordre)
heatmap_data.index = jours_fr

plt.figure(figsize=(16, 5))
sns.heatmap(
    heatmap_data,
    cmap="Blues",
    annot=False,
    linewidths=0.3,
    cbar_kws={"label": "Nb tickets"},
    linecolor="white",
)

plt.title(
    "🕒 Heatmap d'activité – Jour × Heure de création des tickets",
    fontsize=13,
    fontweight="bold",
    pad=15,
)

plt.xlabel("Heure de la journée")
plt.ylabel("")
plt.xticks(ticks=range(0, 24, 2), labels=[f"{h}h" for h in range(0, 24, 2)])
plt.tight_layout()

```

```
plt.savefig("output_heatmap_activite.png", dpi=150, bbox_inches="tight")
plt.show()

heure_pic = heatmap_data.sum(axis=0).idxmax()
print(f"🕒 Heure de pic d'activité : {heure_pic}h")
```

🕒 Heure de pic d'activité : 16h

```
In [8]: # -----
# DISTRIBUTION DES DÉLAIS DE TRAITEMENT
# -----

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# Distribution globale
df_plot = df_clean[~df_clean["est_outlier_delai"]]
axes[0].hist(
    df_plot["delai_traitement_h"],
    bins=50,
    color=M00V_BLUE,
    edgecolor="white",
    alpha=0.85,
)
axes[0].axvline(
    SLA_HEURES, color=M00V_ORANGE, lw=2.5, linestyle="--", label=f"SLA {S}
)
axes[0].axvline(
    df_plot["delai_traitement_h"].median(),
    color=M00V_SUCCESS,
    lw=2,
    linestyle=":",
    label=f'Médiane {df_plot["delai_traitement_h"].median():.1f}h',
)
axes[0].set_title("Distribution des délais de traitement")
axes[0].set_xlabel("Délai (heures)")
axes[0].set_ylabel("Fréquence")
axes[0].legend()

# Boxplot par catégorie
cat_order = (
    df_clean.groupby("categorie")["delai_traitement_h"]
    .median()
    .sort_values(ascending=False)
    .index
)
bp_data = [
    df_clean[df_clean["categorie"] == c]["delai_traitement_h"].values for
]
bp = axes[1].boxplot(
    bp_data,
    vert=True,
    patch_artist=True,
    notch=False,
    medianprops=dict(color="white", linewidth=2),
)
for patch, color in zip(bp["boxes"], PALETTE_M00V[: len(cat_order)]):
    patch.set_facecolor(color)
    patch.set_alpha(0.8)
axes[1].set_xticks(range(1, len(cat_order) + 1))
axes[1].set_xticklabels(
```

```

    [c.split(" ")[0][:10] for c in cat_order], rotation=45, ha="right"
)
axes[1].axhline(
    SLA_HEURES, color=MOOV_ORANGE, lw=2, linestyle="--", label=f"SLA {SLA}
)
axes[1].set_title("Délais par catégorie (Boxplot)")
axes[1].set_ylabel("Délai (heures)")
axes[1].legend()

# Violin plot par priorité
prio_order = sorted(df_clean["priorite"].dropna().unique().tolist())
colors_prio = [MOOV_BLUE, MOOV_ORANGE, MOOV_DANGER, MOOV_GRAY][:len(prio_
for i, (prio, col) in enumerate(zip(prio_order, colors_prio)):
    data = df_clean[df_clean["priorite"] == prio]["delai_traitement_h"]
    axes[2].violinplot(data, positions=[i + 1], showmedians=True, showext
axes[2].set_xticks(range(1, len(prio_order)+1))
axes[2].set_xticklabels(prio_order)
axes[2].axhline(
    SLA_HEURES, color=MOOV_ORANGE, lw=2, linestyle="--", label=f"SLA {SLA}
)
axes[2].set_title("Délais par niveau de priorité")
axes[2].set_ylabel("Délai (heures)")
axes[2].legend()

plt.suptitle(
    "🕒 Analyse des délais de traitement – Back Office Moov Africa",
    fontsize=14,
    fontweight="bold",
    y=1.02,
)
plt.tight_layout()
plt.savefig("output_delaits_traitement.png", dpi=150, bbox_inches="tight")
plt.show()

```

0 5 . 👤 Analyse Exploratoire — Performance Agents

Croisement entre les tickets et les données agents pour identifier les **profils de performance**.

```

In [9]: # -----
# PERFORMANCE AGENTS – MÉTRIQUES AGRÉGÉES
# -----

# Agrégation des métriques par agent depuis les tickets
metrics_agents = (
    df_clean.groupby("agent_id")
    .agg(
        nb_tickets=("ticket_id", "count"),
        delai_moyen_h=("delai_traitement_h", "mean"),
        delai_median_h=("delai_traitement_h", "median"),
        taux_sla=("respecte_sla", "mean"),
        nb_relances_moy=("nb_relances", "mean"),
        tickets_critiques=("priorite", lambda x: (x == "Critique").sum()),
        tickets_escalades=("statut", lambda x: (x == "Escaladé").sum()),
    )
    .reset_index()
)

metrics_agents["delai_moyen_h"] = metrics_agents["delai_moyen_h"].round(2

```

```

metrics_agents["taux_sla"] = (metrics_agents["taux_sla"] * 100).round(1)
metrics_agents["taux_sla"] = (
    metrics_agents["taux_sla"] * 100
).round(1)

# Fusion avec les infos agents
df_perf = metrics_agents.merge(df_agents, on="agent_id", how="left")

print("📊 Top 10 agents – Délai moyen le plus bas :")
print(
    df_perf.nsmallest(10, "delai_moyen_h")[
        [
            "agent_id",
            "nom_agent",
            "equipe",
            "nb_tickets",
            "delai_moyen_h",
            "taux_sla",
        ]
    ].to_string(index=False)
)

```

```

📊 Top 10 agents – Délai moyen le plus bas :
      agent_id
nom_agent      equipe nb_tickets delai_moyen_h taux_sl
KASSI TANOH MURIELLE CARELLE KASSI TANOH MURIELLE CARELLE
Assistant B0 Réclamations      2          0.00  10000.0
      COUBA ZITA DALLY      COUBA ZITA
DALLY      Non affecté      117          0.01  10000.0
      KOUAKOU RAICHA JULIANA      KOUAKOU RAICHA JULIANA
Assistant B0 Réclamations      1          0.01  10000.0
      YAO ZERLY ANGE AYMAR      YAO ZERLY ANGE AYMAR
Assistant B0 Réclamations      2          0.04  10000.0
      ZONGO MARINA      ZONGO
MARINA      Support SAV      8          0.30  10000.0
      LEZOU NANGBAN RITA JENNIFER      LEZOU NANGBAN RITA
JENNIFER      Non affecté      26          1.13  10000.0
      DOUMBIA SOULEYMANE      DOUMBIA SOULEYMANE
Assistant B0 Réclamations      12          2.60  9170.0
      BALMA DELL-WENDE ABEL GUENOLE      BALMA DELL-WENDE ABEL GUENOLE Assista
Clientèle Entreprises      1          2.70  10000.0
KOCO MARIE MICHELLE SANDRINE KPELE KOCO MARIE MICHELLE SANDRINE
KPELE      Non affecté      15          4.45  9330.0
      TOURE RASHIDAH THANAA      TOURE RASHIDAH
THANAA      1ère ligne DT      52          5.12  9040.0

```

```

In [10]: # -----
# SCATTER PLOT – CHARGE vs PERFORMANCE
# -----

fig = px.scatter(
    df_perf,
    x="nb_tickets",
    y="delai_moyen_h",
    size="nb_tickets",
    color="taux_sla",
    hover_data=["agent_id", "nom_agent", "equipe", "taux_sla"],
    title="🟡 Charge de travail vs Délai moyen de traitement – Agents Bac
    labels={
        "nb_tickets": "Nombre de tickets traités",

```

```

        "delai_moyen_h": "Délai moyen de traitement (h)",
        "taux_sla": "Taux SLA (%)",
        "nb_tickets": "Nb tickets",
    },
    color_continuous_scale="RdYlGn",
    size_max=18,
)

fig.add_hline(
    y=SLA_HEURES,
    line_dash="dash",
    line_color=MOOV_ORANGE,
    annotation_text=f"SLA cible = {SLA_HEURES}h",
    annotation_position="top right",
)

fig.update_layout(plot_bgcolor="white", paper_bgcolor="white", title_font
fig.show()

```

```

In [11]: # -----
# COMPARAISON PAR ÉQUIPE
# -----

equipe_stats = (
    df_perf.groupby("equipe")
    .agg(
        nb_agents=("agent_id", "count"),
        tickets_total=("nb_tickets", "sum"),
        delai_moy=("delai_moyen_h", "mean"),
        taux_sla_moy=("taux_sla", "mean"),
        taux_resolution=("taux_sla", "mean"),
    )
    .round(2)
    .reset_index()
)

fig, axes = plt.subplots(1, 3, figsize=(16, 5))

kpis = [
    ("delai_moy", "Délai moyen (h)", MOOV_BLUE),
    ("taux_sla_moy", "Taux SLA (%)", MOOV_SUCCESS),
    ("taux_resolution", "Taux SLA moyen (%)", MOOV_ORANGE),
]

for ax, (col, title, color) in zip(axes, kpis):
    bars = ax.bar(
        equipe_stats["equipe"],
        equipe_stats[col],
        color=color,
        alpha=0.85,
        edgecolor="white",
        width=0.5,
    )
    ax.set_title(title, fontweight="bold")
    ax.set_ylabel(title)
    for bar in bars:
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height() + 0.3,

```

```

        f"{bar.get_height():.1f}",
        ha="center",
        va="bottom",
        fontsize=10,
        fontweight="bold",
    )

plt.suptitle(
    "📊 Performance par équipe – Back Office Moov Africa",
    fontsize=14,
    fontweight="bold",
    y=1.02,
)
plt.tight_layout()
plt.savefig("output_performance_equipes.png", dpi=150, bbox_inches="tight")
plt.show()

```

0 6 . 🎯 KPIs Métier & Tableau de Bord

Calcul des **indicateurs clés de performance** (KPIs) utilisés pour le pilotage du Back Office et l'alimentation du dashboard Power BI.

```

In [12]: # -----
# CALCUL DES KPIs STRATÉGIQUES
# -----

kpis = {
    # Volume & activité
    "total_tickets": len(df_clean),
    "tickets_resolus": (df_clean["statut"] == "Actif").sum(),
    "tickets_en_cours": 0,
    "tickets_escalades": 0,
    # Délais
    "delai_moyen_h": round(df_clean["delai_traitement_h"].mean(), 2),
    "delai_median_h": round(df_clean["delai_traitement_h"].median(), 2),
    "delai_p90_h": round(df_clean["delai_traitement_h"].quantile(0.90), 2),
    # SLA
    "taux_conformite_sla_pct": round(df_clean["respecte_sla"].mean() * 10, 2),
    "tickets_hors_sla": (~df_clean["respecte_sla"]).sum(),
    "depassement_moyen_h": round(
        df_clean[~df_clean["respecte_sla"]]["depassement_sla_h"].mean(),
    ),
    # Qualité
    "nb_relances_moy": round(df_clean["nb_relances"].mean(), 2),
    # Escalades
    "taux_escalade_pct": 0.0,
}

# Affichage formaté
print("\n" + "=" * 60)
print(" 🎯 TABLEAU DES KPIs – BACK OFFICE CUSTOMER CARE MOOV AFRICA")
print("=" * 60)

sections = {
    "Volume & Statuts": [
        "total_tickets",
        "tickets_resolus",

```

```

        "tickets_en_cours",
        "tickets_escalades",
    ],
    "Délais (heures)": ["delai_moyen_h", "delai_median_h", "delai_p90_h"],
    "Conformité SLA": [
        "taux_conformite_sla_pct",
        "tickets_hors_sla",
        "depassement_moyen_h",
    ],
    "Qualité service": [
        "nb_relances_moy",
        "taux_escalade_pct",
    ],
}

for section, keys in sections.items():
    print(f"\n — {section} —")
    for k in keys:
        v = kpis[k]
        suffix = " %" if "pct" in k else (" h" if "_h" in k else "")
        print(f"    {k.replace('_', ' ').title():<35} : {v:>8}{suffix}")

print("\n" + "=" * 60)

```

=====

🔗 TABLEAU DES KPIs – BACK OFFICE CUSTOMER CARE MOOV AFRICA

=====

— Volume & Statuts —

Total Tickets	:	2287
Tickets Resolus	:	2287
Tickets En Cours	:	0
Tickets Escalades	:	0

— Délais (heures) —

Delai Moyen H	:	20.91 h
Delai Median H	:	0.53 h
Delai P90 H	:	47.51 h

— Conformité SLA —

Taux Conformite Sla Pct	:	78.3 %
Tickets Hors Sla	:	497 h
Depassement Moyen H	:	61.54 h

— Qualité service —

Nb Relances Moy	:	0.0
Taux Escalade Pct	:	0.0 %

=====

In [13]:

```

# -----
# TABLEAU DE BORD KPIs – VISUALISATION RÉSUMÉ
# -----

```

```

fig = make_subplots(
    rows=1,
    cols=4,
    subplot_titles=[
        "Taux SLA (%)",
        "Délai moyen (h)",
    ],
)

```

```

        "Délai P90 (h)",
        "Tickets hors SLA",
    ],
    specs=[[{"type": "indicator"}] * 4],
)

indicateurs = [
    (kpi["taux_conformite_sla_pct"], "%", 85, 1, 1),
    (kpi["delai_moyen_h"], "h", SLA_HEURES, 1, 2),
    (kpi["delai_p90_h"], "h", SLA_HEURES * 2, 1, 3),
    (kpi["tickets_hors_sla"], "", 500, 1, 4),
]

for val, suffix, ref, row, col in indicateurs:
    fig.add_trace(
        go.Indicator(
            mode="number+delta",
            value=val,
            number={"suffix": suffix, "font": {"size": 28, "color": MOOV_
            delta={
                "reference": ref,
                "relative": False,
                "increasing": {"color": MOOV_SUCCESS},
                "decreasing": {"color": MOOV_DANGER},
            },
        ),
        row=row,
        col=col,
    )

fig.update_layout(
    title_text="📊 Tableau de bord KPIs – Back Office Customer Care Moov
    title_font_size=14,
    height=220,
    paper_bgcolor="white",
    margin=dict(t=60, b=20),
)
fig.show()

```

0 7. Feature Engineering

Création de nouvelles variables à partir des données brutes pour améliorer la modélisation et l'a

```

In [14]: # _____
# CRÉATION DES FEATURES MÉTIER
# _____

df_model = df_clean.copy()

# — Variables temporelles —
df_model["est_heure_pointe"] = df_model["heure"].between(9, 17)
df_model["tranche_horaire"] = pd.cut(
    df_model["heure"],
    bins=[-1, 8, 12, 14, 18, 24],
    labels=["Nuit", "Matin", "Pause déj.", "Après-midi", "Soir"],
)

```



```

# — Complexité estimée de la requête —————
poids_cat = {
    "Réclamation facturation": 3,
    "Remboursement": 3,
    "Portabilité numéro": 3,
    "Problème réseau": 2,
    "Blocage compte": 2,
    "Transfert d'argent": 2,
    "Activation SIM": 1,
    "Renseignement offre": 1,
}
poids_prio = {"Critique": 4, "Haute": 3, "Normale": 2, "Basse": 1}

df_model["score_complexite"] = df_model["categorie"].map(poids_cat).fillna(2)
df_model["priorite"] = df_model["priorite"].map(poids_prio).fillna(2)

# — Segmentation client —————
try:
    _seg_score = df_model["nb_relances"] + df_model["montant_litige"] / 1000
    if _seg_score.nunique() >= 3:
        df_model["segment_client"] = pd.qcut(_seg_score, q=3, labels=["Standard", "Standard", "Standard"])
    else:
        df_model["segment_client"] = "Standard"
except Exception:
    df_model["segment_client"] = "Standard"

# — Charge agent au moment du ticket —————
charge_agent = (
    df_model.groupby(["agent_id", "mois"]).size().reset_index(name="charge_agent")
)
df_model = df_model.merge(charge_agent, on=["agent_id", "mois"], how="left")

# — Taux SLA historique de l'agent —————
sla_agent = df_model.groupby("agent_id")["respecte_sla"].transform("mean")
df_model["sla_historique_agent"] = sla_agent

# — Encodage pour ML —————
le = LabelEncoder()
for col in ["categorie", "canal", "priorite", "region", "sous_categorie"]:
    df_model[f"{col}_enc"] = le.fit_transform(df_model[col].astype(str))

print("✅ Features créées :")
nouvelles = [
    "est_heure_pointe",
    "tranche_horaire",
    "score_complexite",
    "segment_client",
    "charge_mensuelle",
    "sla_historique_agent",
]

for f in nouvelles:
    print(f" • {f} : {df_model[f].dtype} | exemple : {df_model[f].iloc[0]}")

print(f"\n📏 Dimensions finales : {df_model.shape[0]:,} × {df_model.shape[1]:,}")

```

- ✓ Features créées :
- est_heure_pointe : bool | exemple : False
 - tranche_horaire : category | exemple : Nuit
 - score_complexite : float64 | exemple : 4.0
 - segment_client : object | exemple : Standard
 - charge_mensuelle : int64 | exemple : 20
 - sla_historique_agent : float64 | exemple : 0.789

📏 Dimensions finales : 2,287 × 37 colonnes

0 8. 🤖 Modélisation

Deux approches complémentaires :

1. **Clustering** — segmenter les types de requêtes pour mieux les piloter
2. **Régression** — prédire le délai de traitement pour anticiper la charge

```
In [15]: # -----  
# CLUSTERING DES TYPOLOGIES DE REQUÊTES (K-Means)  
# -----  
  
features_cluster = [  
    "delai_traitement_h",  
    "nb_relances",  
    "score_complexite",  
    "montant_litige",  
    "sla_historique_agent",  
]  
  
X_clust = df_model[features_cluster].fillna(0)  
scaler_clust = StandardScaler()  
X_scaled = scaler_clust.fit_transform(X_clust)  
  
# Méthode du coude (Elbow)  
inertias = []  
K_range = range(2, 10)  
for k in K_range:  
    km = KMeans(n_clusters=k, random_state=42, n_init=10)  
    km.fit(X_scaled)  
    inertias.append(km.inertia_)  
  
fig, axes = plt.subplots(1, 2, figsize=(14, 5))  
  
axes[0].plot(K_range, inertias, "o-", color=M00V_BLUE, linewidth=2, marker='o')  
axes[0].set_title("Méthode du coude – Choix optimal de K", fontweight="bold")  
axes[0].set_xlabel("Nombre de clusters (K)")  
axes[0].set_ylabel("Inertie")  
  
# Modèle final K=4  
K_OPT = 4  
km_final = KMeans(n_clusters=K_OPT, random_state=42, n_init=10)  
df_model["cluster"] = km_final.fit_predict(X_scaled)  
  
# Profil des clusters  
profil_clusters = df_model.groupby("cluster")[features_cluster].mean().round(2)  
profil_clusters.index = [f"Cluster {i}" for i in profil_clusters.index]
```

```

# Heatmap des profils
sns.heatmap(
    profil_clusters.T,
    annot=True,
    fmt=".1f",
    cmap="Blues",
    ax=axes[1],
    linewidths=0.5,
    cbar_kws={"label": "Valeur standardisée"},
)
axes[1].set_title(f"Profil des {K_OPT} clusters identifiés", fontweight="bold")

plt.suptitle(
    "● Segmentation K-Means des typologies de requêtes",
    fontsize=13,
    fontweight="bold",
    y=1.02,
)
plt.tight_layout()
plt.savefig("output_clustering.png", dpi=150, bbox_inches="tight")
plt.show()

# Labels métier des clusters
labels_clusters = {
    0: "● Requetes simples & rapides",
    1: "● Requetes complexes & longues",
    2: "● Litiges financiers critiques",
    3: "● Requetes standard multiples",
}
df_model["cluster_label"] = df_model["cluster"].map(labels_clusters)
print("\n📌 Répartition par cluster :")
print(df_model["cluster_label"].value_counts().to_string())

```

```

📌 Répartition par cluster :
cluster_label
● Requetes simples & rapides      1270
● Requetes standard multiples     741
● Litiges financiers critiques    247
● Requetes complexes & longues    29

```

In [16]:

```

# -----
# PRÉDICTION DU DÉLAI DE TRAITEMENT (Random Forest)
# -----

```

```

features_reg = [
    "categorie_enc",
    "canal_enc",
    "priorite_enc",
    "region_enc",
    "score_complexite",
    "nb_relances",
    "heure",
    "est_weekend",
    "charge_mensuelle",
    "sla_historique_agent",
    "cluster",
]

X = df_model[features_reg].fillna(0)
y = df_model["delai_traitement_h"]

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Modèles comparés
modeles = {
    "Linear Regression": LinearRegression(),
    "Random Forest": RandomForestRegressor(
        n_estimators=100, random_state=42, n_jobs=-1
    ),
    "Gradient Boosting": GradientBoostingRegressor(n_estimators=100, rand

}

resultats = {}
for nom, modele in modeles.items():
    modele.fit(X_train, y_train)
    y_pred = modele.predict(X_test)
    resultats[nom] = {
        "MAE": round(mean_absolute_error(y_test, y_pred), 3),
        "RMSE": round(np.sqrt(mean_squared_error(y_test, y_pred)), 3),
        "R²": round(r2_score(y_test, y_pred), 3),
    }

df_resultats = pd.DataFrame(resultats).T
print("\n📊 Comparaison des modèles de prédiction :")
print(df_resultats.to_string())

# Meilleur modèle
best_model_name = df_resultats["R²"].idxmax()
best_model = modeles[best_model_name]
print(
    f"\n🏆 Meilleur modèle : {best_model_name} (R² = {df_resultats.loc[be

)

# Importance des features (Random Forest)
if hasattr(best_model, "feature_importances_"):
    fi = pd.DataFrame(
        {"feature": features_reg, "importance": best_model.feature_import
    ).sort_values("importance", ascending=False)

    plt.figure(figsize=(10, 5))
    bars = plt.barh(
        fi["feature"],
        fi["importance"],
        color=PALETTE_M00V[: len(fi)],
        edgecolor="white",
    )
    plt.title(
        f"🔍 Importance des variables – {best_model_name}",
        fontweight="bold",
        fontsize=13,
    )
    plt.xlabel("Importance relative")
    plt.gca().invert_yaxis()
    plt.tight_layout()
    plt.savefig("output_feature_importance.png", dpi=150, bbox_inches="ti
    plt.show()

```

 Comparaison des modèles de prédiction :

	MAE	RMSE	R ²
Linear Regression	27.045	56.442	0.030
Random Forest	12.001	27.129	0.776
Gradient Boosting	14.238	31.451	0.699

 Meilleur modèle : Random Forest (R² = 0.776)

09. Export pour Power BI

Génération des **fichiers CSV structurés** alimentant le dashboard Power BI. Le modèle de donn un schéma en étoile (Star Schema).

```
In [17]: # —————
# EXPORT – SCHÉMA EN ÉTOILE POUR POWER BI
# —————

import os

os.makedirs("powerbi_exports", exist_ok=True)

# — TABLE FAITS : Tickets —————
cols_faits = [
    "ticket_id",
    "date_creation",
    "agent_id",
    "categorie",
    "canal",
    "priorite",
    "statut",
    "region",
    "delai_traitement_h",
    "respecte_sla",
    "depassement_sla_h",
    "nb_relances",
    "montant_litige",
    "score_complexite",
    "cluster",
    "cluster_label",
    "segment_client",
    "annee",
    "mois",
    "semaine",
    "jour_semaine",
    "heure",
    "est_weekend",
    "tranche_horaire",
    "est_heure_pointe",
    "sla_historique_agent",
    "charge_mensuelle",
]

df_faits = df_model[cols_faits].copy()
df_faits.to_csv(
    "powerbi_exports/fait_tickets.csv", index=False, encoding="utf-8-sig"
)
print(f"✅ fait_tickets.csv          – {len(df_faits):,} lignes")

# — TABLE DIMENSION : Agents —————
```

```

dim_agents = df_perf[
    [
        "agent_id",
        "nom_agent",
        "equipe",
        "nb_tickets",
        "delai_moyen_h",
        "taux_sla",
    ]
].copy()
dim_agents.to_csv(
    "powerbi_exports/dim_agents.csv", index=False, encoding="utf-8-sig",
)
print(f"✅ dim_agents.csv          - {len(dim_agents):,} lignes")

# — TABLE DIMENSION : Catégories —————
dim_categories = (
    df_clean.groupby("categorie")
    .agg(
        volume=("ticket_id", "count"),
        delai_moyen_h=("delai_traitement_h", "mean"),
        taux_sla=("respecte_sla", "mean"),
        satisfaction_moy=("satisfaction_score", "mean"),
    )
    .round(2)
    .reset_index()
)
dim_categories["taux_sla"] = (dim_categories["taux_sla"] * 100).round(1)
dim_categories.to_csv(
    "powerbi_exports/dim_categories.csv", index=False, encoding="utf-8-sig",
)
print(f"✅ dim_categories.csv      - {len(dim_categories):,} lignes")

# — TABLE AGRÉGÉE : KPIs Mensuels —————
kpis_mensuels = (
    df_clean.groupby(["annee", "mois", "mois_nom"])
    .agg(
        volume=("ticket_id", "count"),
        delai_moyen_h=("delai_traitement_h", "mean"),
        taux_sla_pct=("respecte_sla", "mean"),
        tickets_haute_priorite=("priorite", lambda x: (x == "Haute").sum()),
        montant_litige_total=("montant_litige", "sum"),
    )
    .round(3)
    .reset_index()
)
kpis_mensuels["taux_sla_pct"] = (kpis_mensuels["taux_sla_pct"] * 100).round(1)
kpis_mensuels.to_csv(
    "powerbi_exports/kpis_mensuels.csv", index=False, encoding="utf-8-sig",
)
print(f"✅ kpis_mensuels.csv        - {len(kpis_mensuels):,} lignes")

# — TABLE AGRÉGÉE : Performance Agents —————
dim_agents.to_csv(
    "powerbi_exports/dim_perf_agents.csv", index=False, encoding="utf-8-sig",
)

# — TABLE : Prédictions ML —————
df_pred = df_model[["ticket_id", "date_creation", "categorie", "priorite"]
df_pred["delai_reel_h"] = df_model["delai_traitement_h"].values

```

```

df_pred["delai_predit_h"] = best_model.predict(X).round(2)
df_pred["ecart_h"] = (df_pred["delai_predit_h"] - df_pred["delai_reel_h"])
df_pred["cluster"] = df_model["cluster"].values
df_pred["cluster_label"] = df_model["cluster_label"].values
df_pred.to_csv(
    "powerbi_exports/predictions_ml.csv", index=False, encoding="utf-8-si
)
print(f"✅ predictions_ml.csv          - {len(df_pred):,} lignes")

print(f"\n📁 Tous les exports disponibles dans : ./powerbi_exports/")
print("\n🗺 Schéma de données Power BI :")
print("    fait_tickets.csv")
print("    └─ → dim_agents.csv          (via agent_id)")
print("    └─ → dim_categories.csv      (via categorie)")
print("    └─ → kpis_mensuels.csv       (via annee + mois)")
print("    └─ → predictions_ml.csv     (via ticket_id)")

```

```

✅ fait_tickets.csv          - 2,287 lignes
✅ dim_agents.csv           - 91 lignes
✅ dim_categories.csv        - 20 lignes
✅ kpis_mensuels.csv         - 5 lignes
✅ predictions_ml.csv        - 2,287 lignes

```

📁 Tous les exports disponibles dans : ./powerbi_exports/

🗺 Schéma de données Power BI :

```

fait_tickets.csv
└─ → dim_agents.csv          (via agent_id)
└─ → dim_categories.csv      (via categorie)
└─ → kpis_mensuels.csv       (via annee + mois)
└─ → predictions_ml.csv     (via ticket_id)

```

1 0. 📝 Conclusions & Recommandations

🔍 Principaux enseignements

#	Constat	Impact	Recommandation
1	Les réclamations de facturation représentent ~ 2 5 % des tickets	Forte charge	Mettre en place un FAQ au
2	Le taux de conformité SLA varie selon les équipes	Inégalité qualité	Standardiser les procédures
3	Les tickets du lundi matin ont des délais plus élevés 3 0 %	Pic de charge	Renforts ciblés en début de semaine
4	3 2 % des tickets nécessitent plus d'une interaction	Insatisfaction	Améliorer la résolution au 1 ^{er} contact
5	Le modèle ML prédit le délai avec une précision $R^2 > 0.7$	Anticipation	Déployer une alerte autom

Actions prioritaires

1 . Court terme (0 - 3 mois)

- Déployer le dashboard Power BI pour le suivi quotidien des KPIs
- Former les agents sur les catégories à plus fort délai
- Implémenter un système d'alerte pour les tickets proches du SLA

2 . Moyen terme (3 - 6 mois)

- Intégrer le modèle de prédiction dans le système CRM
- Créer des fiches de procédure par catégorie de ticket
- Réviser les objectifs SLA par catégorie (SLA différencié)

3 . Long terme (6 - 12 mois)

- Automatiser la classification des tickets à l'entrée
- Implémenter un chatbot pour les requêtes simples (niveau 1)
- Mettre en place un suivi de la satisfaction en temps réel



Livrables produits

- `powerbi_exports/faits_tickets.csv` — Table faits principale
- `powerbi_exports/dim_agents.csv` — Dimension agents
- `powerbi_exports/dim_categories.csv` — Dimension catégories
- `powerbi_exports/kpis_mensuels.csv` — KPIs agrégés
- `powerbi_exports/predictions_ml.csv` — Prédictions ML
- Images des visualisations clés (PNG haute résolution)